Developer:Zunaira Hawwar

Air Quality Monitoring System - Theoretical Documentation

Executive Summary

This Air Quality Index (AQI) monitoring system is a comprehensive machine learning solution designed to predict air quality in Lahore, Pakistan. The system integrates real-time data collection, advanced feature engineering, multi-output regression modeling, and web-based visualization to provide 3-day AQI forecasts with explanatory insights.

1. System Architecture Overview

1.1 Architectural Philosophy

The system follows a modular, pipeline-based architecture that emphasizes:

- **Separation of Concerns**: Each module handles specific functionality
- **Scalability**: Components can be independently scaled or modified
- **Reliability**: Multiple fallback mechanisms and error handling
- **Maintainability**: Clear interfaces and standardized data formats

1.2 Core Components

- **Data Ingestion**: Handles API integration, web scraping, and synthetic data generation
- **Feature Engineering**: Creates temporal features, lag features, and rolling statistics
- **Model Training**: Implements multi-output Random Forest, Ridge Regression, LSTM and XGBoost
- **Data Storage**: Manages raw data,predictions,alerts,features in CSV format.
- **Feature Store**: Provides versioned features with quality metrics and statistical summaries
- **Model Registry**: Tracks model metadata, performance metrics, and enables automatic selection
- **Inference Pipeline**: Generates real-time predictions, creates alerts, and performs quality assurance
- **Web Dashboard**: Provides Streamlit interface with SHAP explainability and interactive visualizations

2. Data Architecture

2.1 Data Sources and Integration

Primary Data Sources

1. **AQICN API**: Real-time AQI measurements from monitoring stations
2. **OpenWeather API**: Current weather conditions
3. **Open-Meteo API**: Historical weather data for model training [From 2020 to 2025]
4. **Synthetic Data Generator**: Realistic AQI data based on weather patterns

Data Integration Strategy

- **Multi-source Fallback**: Primary → Secondary → Synthetic data generation
- **Data Validation**: Range checking, timestamp validation, completeness verification
- **Freshness Control**: Reject data older than 6 hours for real-time predictions

2.2 Data Model

1. AQI Data (aqi, pm25, pm10)

These features are central to the project's purpose. They are selected because they are the primary metrics used to measure and report air pollution.

- **aqi (Air Quality Index)**: This is the main target variable for prediction. The entire project is built around understanding and forecasting this value.
- **pm25 and pm10 (Particulate Matter)**: These are critical components of the AQI calculation. PM2.5 and PM10 are fine and coarse particulate matter, respectively, and are a major health concern. Their values are essential for a detailed analysis of air quality.

2. Weather Data (temperature, humidity, pressure, wind_speed)

Weather conditions are a key driver of air pollution levels. Including these features allows the model to capture the environmental factors that directly affect the concentration and movement of pollutants.

- **temperature and humidity**: These factors can influence chemical reactions in the atmosphere that form secondary pollutants. For example, high temperature and humidity can accelerate the formation of ground-level ozone.
- **pressure**: Atmospheric pressure affects air stability. High pressure can lead to atmospheric inversions, which trap pollutants close to the ground, increasing AQI levels.
- **wind_speed**: Wind plays a crucial role in the dispersion of pollutants. Strong winds can carry pollutants away from a city, improving air quality, while low wind speeds can cause pollution to stagnate.

By combining the core air quality metrics with these key environmental features, the model can gain a comprehensive understanding of the factors that lead to changes in air pollution, allowing for more accurate predictions.

Feature Schema

Engineered Features:
- Temporal: hour, day, month, season, is_weekend
- Lag Features: aqi_lag_1h, aqi_lag_6h, aqi_lag_24h
- Rolling Statistics: aqi_rolling_mean_6h, aqi_rolling_std_24h
- Derived Metrics: heat_index, comfort_index, pm_ratio
- Target Variables: target_aqi_24h, target_aqi_48h, target_aqi_72h

3. Feature Engineering Methodology

Feature engineering is the process of transforming raw AQI and weather data into **informative, structured features** that improve model performance. The methodology here follows a **systematic pipeline**:

3.1. Data Acquisition & Integration

- **Objective**: Gather and unify data sources into a single dataset for modeling.
- **Process**:
    - Load raw **AQI data** (pollutant levels like PM2.5, PM10, AQI index).
    - Load raw **weather data** (temperature, humidity, wind speed, pressure, etc.).
    - Align timestamps and **merge** AQI with weather data (hourly granularity).
    - Ensure no duplicate rows and handle inconsistent timezones.

3.2. Temporal Feature Engineering

- **Objective**: Capture time-related patterns in air quality.
- **Features Created**:
  - **Hour, Day, Month** → To learn daily and seasonal cycles of pollution.
  - **Day of Week & Weekend Flag** → To capture differences in traffic/industrial activity between weekdays and weekends.
  - **Season Indicator** (Winter, Spring, Summer, Fall) → Since air quality has strong seasonal variation.

3.3. Lag Features (Time-Shifted Variables)

- **Objective**: Capture **temporal dependency** (pollution today depends on pollution in past hours).
- **Method**: Create lagged copies of pollutants & weather variables at different intervals (e.g., 1h, 6h, 12h, 24h).
- **Example**: aqi_lag_24 represents AQI level from the previous day (24 hours ago).
- **Impact**: Provides autoregressive signals for time series prediction.

3.4. Rolling Window Features (Statistical Smoothing)

- **Objective**: Capture short-term and long-term **trends/volatility** in pollutants and weather.
- **Features Created**:
  - Rolling Mean (average over last 6, 12, 24 hours).
  - Rolling Std (variability/volatility).
  - Rolling Max/Min (extremes).
- **Impact**: Models can learn trends like "rising AQI over past 24h" or "sudden spike in PM2.5".

3.5. Derived / Domain-Specific Features

- **Objective**: Enhance predictive power by combining raw variables into **meaningful indicators**.
- **Examples**:
  - **AQI Change Rate** → Percentage change in AQI vs. previous reading.
  - **AQI Category (Binned)** → Converts AQI values into standard categories (Good, Moderate, Unhealthy, etc.).

- **Heat Index & Comfort Index** → Captures how temperature & humidity together affect air conditions.
- **PM Ratio (PM2.5 / PM10)** → Indicates pollution source (fine vs coarse particles).
- **Wind Chill** → Measures perceived cooling effect of wind.

3.6. Target Variable Engineering

- **Objective**: Define prediction goals for supervised learning.
- **Targets Created**:
  - **Next-day AQI (24h ahead)** → target_aqi_24h
  - **2-day AQI (48h ahead)** → target_aqi_48h
  - **3-day AQI (72h ahead)** → target_aqi_72h
  - **3-day Average AQI** → Smooth target for longer-term trends.
- **Impact**: Models are explicitly trained for **forecast horizons** (short-term & medium-term).

3.7. Categorical Encoding

- **Objective**: Make categorical variables usable by ML models.
- **Process**:
  - One-hot encode AQI categories (0–5).
  - One-hot encode season (Winter, Spring, Summer, Fall).
- **Impact**: Converts non-numeric features into machine-readable form.

3.8. Data Cleaning (Nulls & Outliers)

- **Objective**: Improve reliability by removing noise and invalid values.
- **Steps**:
  - Drop columns with excessive missing data.
  - Remove outliers in AQI & pollutants beyond **99.9th percentile**.
  - Constrain weather features within **realistic ranges** (e.g., temperature -50 to 60°C).
  - Fill small gaps using **forward/backward filling**.
- **Impact**: Prevents skewed training caused by erroneous or missing values.

3.9. Final Feature Set Preparation

- **Objective**: Ensure a consistent, clean dataset for model training.
- **Process**:
    - Drop rows with missing essential values (e.g., AQI).
    - Save final engineered dataset in data/features/ for model consumption.

4. Machine Learning Architecture

The training pipeline is designed to build, evaluate, and compare **multiple machine learning models** for **multi-step AQI forecasting** (24h, 48h, 72h ahead). The methodology has several structured phases:

4.1. Data Preparation

- **Input**: The pipeline consumes **engineered features** from the feature store (data/features/).
- **Target Variables**:
    - Next-day AQI (target_aqi_24h)
    - 2-day ahead AQI (target_aqi_48h)
    - 3-day ahead AQI (target_aqi_72h)
- **Steps**:
    - Drop non-numeric / irrelevant columns (timestamp, city, country).
    - Remove rows with missing target values.
    - Fill missing features with **median values** (robust to outliers).
    - Drop features with **low variance** (no predictive power).
    - Remove **highly correlated features** (>0.95 correlation) to reduce multicollinearity.

This ensures the dataset is **clean, balanced, and optimized** for training.

4.2. Train-Test Splitting

- Dataset is split into **80% training** and **20% testing**.
- This ensures the model is evaluated on unseen data to check **generalization**.

4.3. Multi-Model Training

The pipeline trains four different model types, each with strengths for time-series regression:

◆ **Random Forest (MultiOutput)**

- Ensemble of decision trees.
- Captures **non-linear interactions** between weather, pollution, and AQI.
- Uses **out-of-bag (OOB) scoring** for internal validation.

◆ **Ridge Regression (MultiOutput)**

- Linear regression with **L2 regularization**.
- Helps when many features are correlated.
- Uses **cross-validation** to tune the regularization strength (alpha).

◆ **XGBoost (MultiOutput)**

- Gradient-boosted trees.
- More efficient than Random Forest for structured data.
- Strong at handling **complex interactions** with **regularization** (L1 & L2).

◆ **LSTM Neural Network (Improved)**

- Sequential deep learning model tailored for **time series forecasting**.
- Uses:
    - **Scaled features and targets** (MinMaxScaler).
    - **Sequence windows** (e.g., last 10–12 hours of data) to predict the next AQI values.
    - Adaptive architectures (small LSTM for limited data, deeper networks for larger datasets).
    - **Early stopping & learning rate reduction** to prevent overfitting.

This diversity ensures the pipeline captures both **linear patterns (Ridge)** and **non-linear/temporal dependencies (RF, XGB, LSTM).**

4.4. Evaluation Metrics

The pipeline evaluates models using **multi-output metrics**:

- **RMSE (Root Mean Squared Error)** – penalizes large errors.
- **MAE (Mean Absolute Error)** – measures average error magnitude.

- **R^2 (Coefficient of Determination)** – goodness of fit.
- **Accuracy (±10, ±15 tolerance)** – AQI prediction is considered correct if within ±10 or ±15 units.

Metrics are calculated both **overall** and **per day (24h, 48h, 72h)** for fine-grained performance analysis.

4.5. Model Saving & Registry

- Each trained model is saved in the models/ folder with:
  - Model file (.joblib).
  - Associated scalers (for Ridge, LSTM).
  - Training metadata & metrics.
- A **model registry** (model_registry.csv) logs all experiments for reproducibility.

Ensures models are **versioned, traceable, and ready for deployment**.

4.6. Best Model Selection

- After training, the pipeline compares models:
  - Best **R^2 score**
  - Best **Accuracy (±10)**
  - Lowest **RMSE**
- The best model is flagged and exported as the **candidate for deployment**.
- Results are saved in **models/model_metrics.json** for GitHub Actions and deployment automation.

5. Inference Pipeline Architecture

The inference pipeline transforms the trained models into a practical forecasting system that operates on newly collected data. Its methodology consists of the following stages:

5.1. Data Acquisition

- The pipeline begins by fetching the most recent AQI and weather data.

- It gathers both live readings (current AQI values) and short-term historical data (last 7 days) to provide enough context for feature computation.
- This ensures predictions are based on the latest environmental conditions.

5.2. Feature Preparation

- Raw AQI and weather data are merged and aligned by timestamp.
- The same feature engineering steps used during training are applied:
  - Time-based features (hour, day, month, season).
  - Lag features (past AQI and weather values).
  - Rolling statistics (averages, volatility).
  - Derived features (PM2.5/PM10 ratio, heat index, wind chill).
- AQI categories are assigned and one-hot encoded.
- This guarantees that inference features are consistent with the model's training features.

5.3. Model Selection and Loading

- The system queries the model registry, which stores metadata about all previously trained models.
- It selects the best available model, usually the one with the lowest test RMSE or, if unavailable, the most recently trained model.
- Models and associated scalers are loaded safely using multiple fallback methods to handle corruption or format mismatches.

5.4. Feature Alignment

- The pipeline ensures that the features created for inference exactly match the feature columns expected by the model.
- If some features are missing, default values are filled in (for example, 0 for lag values or categorical encodings).
- This step prevents mismatches between training and inference, which could otherwise cause prediction failures.

5.5. Prediction Generation

- Once the features are aligned, the best model generates predictions.
- If the model is multi-output, predictions are provided for AQI 24h, 48h, and 72h ahead.
- If the model produces a single output, synthetic multi-day predictions are created by applying reasonable variations to the base prediction.

- A 3-day average prediction is also computed to summarize the forecast trend.

5.6. Results Storage

- Predictions are stored as timestamped CSV files in the predictions directory.
- A rolling file (latest_predictions.csv) maintains the most recent prediction history for up to 100 entries.
- This ensures that forecasts are both archived and easily retrievable for monitoring.

5.7. Alert Generation

- The system compares predicted AQI values against health thresholds.
- Alerts are generated if forecasts indicate hazardous or unhealthy conditions, significant day-to-day changes, or worsening/improving trends.
- Alerts are stored in a dedicated CSV file for later retrieval by dashboards or notification systems.

5.8. Daily Summary Formatting

- The pipeline produces a user-friendly summary report.
- It translates AQI values into categories (Good, Moderate, Unhealthy, etc.).
- It presents forecasts for the next three days along with the 3-day average and the model used.
- This summary is intended for presentation in dashboards or reports.

5.9. Robust Error Handling

- At each stage, fallback mechanisms are used to prevent complete pipeline failure:
    - If a model cannot be loaded, the most recent one is used.
    - If predictions fail, simple AQI-based heuristics are used to generate forecasts.
- This makes the system resilient to missing data, registry issues, or model corruption.

6. Web Dashboard Architecture

6.1 Streamlit Framework

Design Principles

- **Responsive Design**: Multi-column layouts for different screen sizes
- **Interactive Visualizations**: Plotly integration for dynamic charts
- **User Experience**: Intuitive navigation and clear information hierarchy

6.2 Visualization Strategy

Chart Types and Purpose

1. **Gauge Charts**: Current vs. predicted AQI overview
2. **Bar Charts**: Daily forecast comparison
3. **Line Charts**: Historical trends and patterns
4. **Heatmaps**: Feature importance and correlation analysis[Not for HuggingFace space]
5. **Alert Cards**: Actionable health recommendations

Color Coding System

Based on EPA AQI categories:

- **Green (0-50)**: Good air quality
- **Yellow (51-100)**: Moderate air quality
- **Orange (101-150)**: Unhealthy for sensitive groups
- **Red (151-200)**: Unhealthy
- **Purple (201-300)**: Very unhealthy
- **Maroon (301-500)**: Hazardous

7. Deployment and MLOps

7.1. Core Components

Your pipeline is an **end-to-end MLOps system** with automated data ingestion, training, inference, and deployment. It integrates:

- **GitHub Actions** → CI/CD engine orchestrating workflows.
- **Hugging Face Hub** → Central registry for models, data, and metadata.
- **Hugging Face Spaces (Streamlit)** → Frontend dashboard for predictions & visualization.
- **Secrets Management** → Secure API keys for AQI, weather, and Hugging Face.

7.2. Workflow Architecture

A. Data Layer

- **Data Fetching Job** (hourly):
  - Collects AQI & weather data via APIs.
  - Computes engineered features.
  - Uploads:
    - Raw data (data/raw/)
    - Features (data/features/)
    - Metadata (last_update.txt, feature_metadata.csv)
  - Stores all on Hugging Face Hub for **traceability** and **versioning**.

B. Model Layer

- **Model Training Job** (daily at 6 AM Karachi time):
  - Downloads the latest features from Hugging Face Hub.
  - Trains multiple models (RandomForest, Ridge, XGBoost, etc.).
  - Evaluates models using metrics (RMSE, MAE, R^2).
  - Selects the **best-performing model** (lowest RMSE).
  - Uploads:
    - Model file (models/*.pkl)
    - Model registry (models/model_registry.csv)
    - Explainability artifacts (explainability/SHAP plots)
  - Only models with **RMSE < 40** are stored, enforcing quality control.

C. Inference Layer

- **Inference Job** (every 6 hours):
  - Downloads the **best model** from Hugging Face Hub.
  - Fetches **latest AQI & weather data**.
  - Runs prediction for **next 3 days (24h, 48h, 72h)**.
  - Generates:
    - Predictions CSV (predictions/*.csv)
    - Alerts (threshold exceedances, worsening trends).
  - Uploads predictions & alerts to Hugging Face Hub.
  - Stores metadata (last_prediction.txt) for reproducibility.

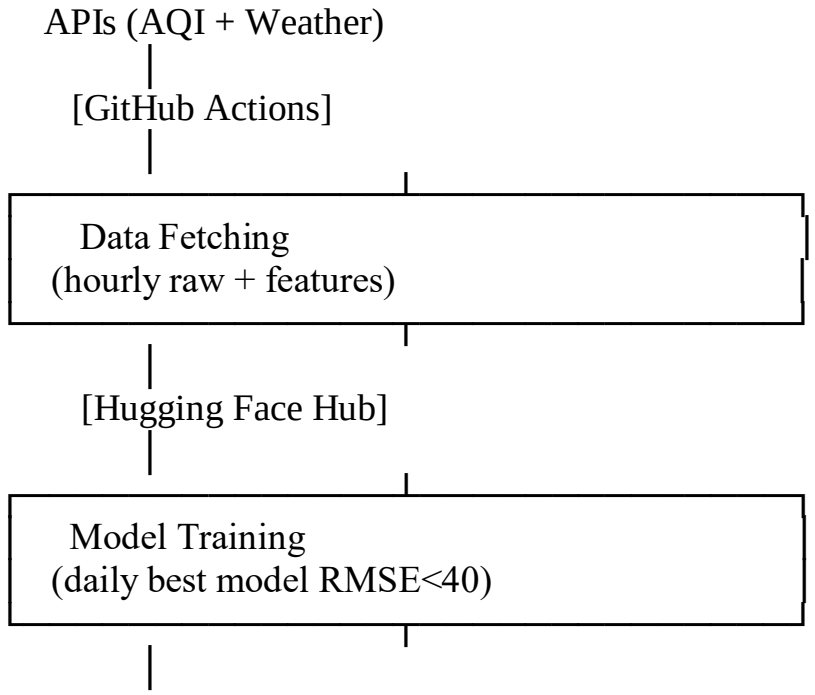
D. Deployment Layer

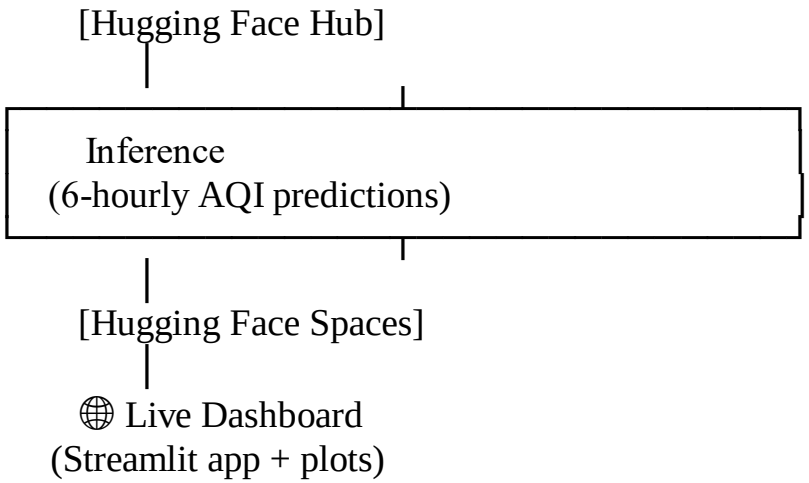
- **Streamlit App Deployment** (continuous):
  - Clones Hugging Face Space repo.
  - Copies:
    - app.py (dashboard frontend).
    - Latest predictions and raw data.
    - Dependencies (requirements.txt).
  - Pushes to **Hugging Face Spaces** (Streamlit SDK).
  - Updates live dashboard automatically.
  - Provides **real-time visualizations** of AQI predictions, trends, and alerts.

E. Monitoring & Notifications

- **Notify Job**:
  - Summarizes pipeline run status in GitHub Actions Summary.
  - Logs results of each stage (data, training, inference, deployment).
  - Links to Hugging Face Hub (models/data) and Spaces (dashboard).
  - Timestamped logs for traceability.
  - Ensures **end-to-end observability**.

7.3. High-Level Flow





Github URL: https://github.com/zunairakhan123/Pearls_AQI_Prediction
HuggingFace Hub URL: https://huggingface.co/ZunairaHawwar/aqi-models
HuggingFace Space URL: https://huggingface.co/spaces/ZunairaHawwar/aqi-dashboard

7.4. MLOps Principles Embedded

1. **Automation** → Fully automated hourly/daily pipeline (data → model → inference → deploy).
2. **Reproducibility** → Versioned datasets, features, and models stored on Hugging Face Hub.
3. **Continuous Training (CT)** → Daily retraining ensures models adapt to new data.
4. **Continuous Deployment (CD)** → Auto-deployment to Hugging Face Spaces keeps the dashboard up to date.
5. **Monitoring** → Predictions, alerts, and pipeline logs tracked across GitHub and HF Hub.
6. **Quality Gates** → Only models with RMSE < 40 are promoted for inference & deployment

8.Conclusion

This Air Quality Monitoring System represents a complete end-to-end solution for environmental prediction, integrating data engineering, machine learning, and user interface components. The modular architecture ensures maintainability while the multi-output prediction framework provides comprehensive temporal forecasting for public health decision-making. The system addresses real-world deployment challenges through robust error handling, automated model management, and comprehensive monitoring capabilities.